COMP70114 – Machine Learning Systems and Hardware
Coursework: CUDA Kernel Development

1 Introduction: Purpose & Learning Outcomes

In Lectures 3 and 4, we explored GPU architecture, GPGPU programming, and the importance of custom kernel implementations in machine learning systems. To further reinforce your understanding and enhance your programming skills, this coursework requires you to design and implement two CUDA kernels corresponding to two common deep learning layers. By completing this coursework, you will gain practical experience in:

- Developing custom CUDA kernels.
- Profiling and debugging with NVIDIA Nsight.
- Performing performance analysis and applying optimization strategies.

Given the diverse backgrounds of students in this course, the difficulty of the assignment has been kept manageable. However, for those seeking an additional challenge, we encourage exploring hard GPU programming exercises such as those on *LeetGPU*, or studying high-quality open-source ML systems like `vllm`, `SGLang` or `nano-vllm`.

2 Question Set

The coursework directory can be downloaded here. It has the following structure:

```
mlsys_coursework/
|-- part1/
|   |-- cpu_bn_inference_ref.cu
|   |-- gpu_bn_inference.cu
|   |-- solve.h
|   |-- test_bn.cu
|   |-- test_script.sh
|-- part2/
    |-- cpu_dwconv2d_inference_ref.cu
    |-- gpu_dwconv2d_inference.cu
    |-- solve.h
    |-- test_dwconv2d.cu
    |-- test_script.sh
```

You are required to complete `gpu_bn_inference.cu` and `gpu_dwconv2d_inference.cu` in each part. Please adhere to the following implementation requirements:

- Do not use any external libraries.

- Do not modify the provided function signatures in `solve.h`. The only files you can modify are `gpu_bn_inference.cu` and `gpu_dwconv2d_inference.cu`
- The final result must be written to the `y` array.
- Run `bash test_script.sh` to evaluate the correctness of simple test cases.

2.1 Part 1: Batch Normalization

Implement **Batch Normalization (inference, forward-only)** for tensors in **NCHW** layout. Given:

- Input tensor $x \in \mathbb{R}^{N\times C\times H\times W}$,
- Per-channel scaling parameters $\gamma, \beta \in \mathbb{R}^{C}$,

compute the normalized output $y \in \mathbb{R}^{N\times C\times H\times W}$ by

$$y[n,c,h,w] = \gamma[c] \cdot \frac{x[n,c,h,w] - \mu_c}{\sqrt{\sigma_c^2 + \varepsilon}} + \beta[c],$$

where

$$\mu_c = \frac{1}{NHW} \sum_{n=0}^{N-1} \sum_{h=0}^{H-1} \sum_{w=0}^{W-1} x[n,c,h,w], \qquad \sigma_c^2 = \frac{1}{NHW} \sum_{n,h,w} \big(x[n,c,h,w] - \mu_c\big)^2.$$

The reference CPU implementation can be found in `cpu_bn_inference_ref.cu`.

Constraints

$$1 \le N \le 1024, \quad 1 \le C \le 8192, \quad 1 \le H, W \le 4096.$$

2.2 Part 2: 2D Depthwise Convolution

Implement **2D depthwise convolution (inference, forward-only)** for **NCHW** input with one filter per input channel. Given:

- Input $x \in \mathbb{R}^{N\times C\times H\times W}$,
- Depthwise kernel $k \in \mathbb{R}^{C\times 1\times K_H\times K_W}$,
- Stride (s_h, s_w) and padding (p_h, p_w),

compute $y \in \mathbb{R}^{N\times C\times O_H\times O_W}$ by

$$y[n,c,oh,ow] \;=\; \sum_{kh=0}^{K_H-1} \sum_{kw=0}^{K_W-1} x\big[n,c,\; oh\cdot s_h - p_h + kh,\; ow\cdot s_w - p_w + kw\big] \cdot k[c,0,kh,kw].$$

The reference CPU implementation can be found in `cpu_dwconv2d_inference_ref.cu`.

Constraints

$1 \leq N \leq 256, \quad 1 \leq C \leq 2048, \quad 1 \leq H, W \leq 4096, \quad 1 \leq K_H, K_W \leq 31, \quad K_H \leq H+2p_h,\ K_W \leq W+2p_w.$

3 Evaluation Criteria and Deliverables

This is an **individual project**. Each student must submit their own implementation and report.

3.1 Marking Scheme

This coursework contributes **20 marks** towards your final grade. It consists of two parts, each worth **10 marks**, evaluated as follows:

- **Correctness (5 marks per part):** Your implementation must pass all correctness tests using the provided `test_script.sh`.
- **Performance (5 marks per part):** Marks will be awarded based on the performance of your CUDA kernels, including achieved throughput and efficient memory usage.

3.2 Deliverables

You must submit the following items via the course submission portal by **Thursday, 27th November at 19:00**:

- **Source Code (ZIP archive):**
  - Submit a ZIP file named `mlsys25_YourCollegeID.zip` with the following two files
    1. `gpu_bn_inference.cu`
    2. `gpu_dwconv2d_inference.cu`
- **PDF Report:** A report of at most **2 pages** (covering both parts combined). Use the 11pt font size and the default margin settings from `\documentclass[11pt]{article}`. It should include:
  - A screenshot showing correctness and performance metrics by submitting your code in our evaluation website. We will re-run your submitted code to cross-validate the results.
  - A brief explanation of your implementation, including any optimizations applied (e.g., memory coalescing, loop unrolling, use of shared memory, etc.).

4 GPU Access and Evaluation Platform

4.1 Lab GPUs for Code Development

This coursework does not involve GPU-intensive training or large-scale inference. You may use any of the laboratory workstations (Table 1) for code development and debugging. We recommend using **CUDA 12.9**, which should already be available on most of these machines.

Follow the steps below to connect to the lab GPU machines:

Host Name	GPU Type
gpu31.doc.ic.ac.uk	NVIDIA GeForce RTX 4080
gpu32.doc.ic.ac.uk	NVIDIA GeForce RTX 4080
gpu33.doc.ic.ac.uk	NVIDIA GeForce RTX 4080
gpu34.doc.ic.ac.uk	NVIDIA GeForce RTX 4080
gpu35.doc.ic.ac.uk	NVIDIA GeForce RTX 4080
gpu36.doc.ic.ac.uk	NVIDIA GeForce RTX 4080

Table 1: Available lab workstations and corresponding GPU types.

1. **Connect to the College Network:** Access requires either being connected to the college network or setting up SSH.

2. **Download Coursework and File Transfer:** Download the ZIP file from this link and unzip it to your desired location (e.g., `/path/to/mlsys_coursework`). To transfer the file to a GPU lab machine (e.g., `gpu31`), use the following `scp` command in your terminal:

```
scp -r /path/to/mlsys_coursework college_id@gpu31.doc.ic.ac.uk:~/.
```

3. **Login to a Lab GPU Machine:** Follow the instructions provided here to set up SSH access. Then log in to a GPU machine (e.g., `gpu31`) using the following command:

```
ssh college_id@gpu31.doc.ic.ac.uk
```

4. **Environmental Setup:** To set up the **nvcc** environment for this session, run the following command before running the code, or add the following under `~/.bashrc`:

```
export PATH="/vol/cuda/12.9.0/bin/:$PATH"
export LD_LIBRARY_PATH="/vol/cuda/12.9.0/lib64:$LD_LIBRARY_PATH"
```

5. **Coding and Debugging:** Implement your CUDA kernels in `gpu_bn_inference.cu` and `gpu_dwconv2d_inference.cu`, then run the provided test script to evaluate correctness and performance.

4.2 Evaluation Platform

Since lab workstations do not have permission to access the program counter, profiling with **NVIDIA Nsight Compute (NCU)** must be performed on a dedicated workstation. We provide access to an evaluation platform equipped with NVIDIA RTX 5090 GPUs, Nsight Compute and Docker-based testing environments. Please use these machines **only when performing NCU profiling or final performance evaluation** to ensure fair access for all users. Follow the steps below to set up your access:

1. **Login and change password:** Access evaluation platform using this website If you are off–campus, please connect by setting up SSH key. Your account name corresponds to your College login, and the default password is `letmein`. Please change your password immediately after your first login to secure your account. If you encounter any issues or have questions, please raise them in this EdStem thread.

2. **Evaluating your code:** Click on each problem to enter its page, and submit your code there for performance and correctness evaluation. Please expect some queuing delays during peak periods.

3. **Transferring NCU Report for Profiling:** After successfully running your code, NCU and Nsys reports will be available for download under "My Submission Records" → "View Log." You can download the report to your desktop or laptop and open it with Nsight Compute for performance profiling. To install Nsight Compute, follow the instructions at this link.

5 Logistics

5.1 Submission Deadline and Late Policy

The submission deadline is **Thursday, 27th November at 19:00**. The submission should be uploaded via Scientia. Coursework submitted up to one day after the assessment deadline (date and time) will be marked but capped at the passmark. Work submitted more than one day late will not be accepted as a valid attempt and mark of zero will be recorded.

5.2 Plagiarism

Plagiarism of any form is strictly prohibited. All submitted work must be entirely your own. We employ multiple mechanisms to detect plagiarism, including cross-submission comparison, similarity checking, and automated code pattern analysis. Any detected case will be reported and penalised in accordance with the Department's academic integrity policies. The use of large language models (LLMs) or other code-generation tools to produce solutions for this coursework is **not permitted**.